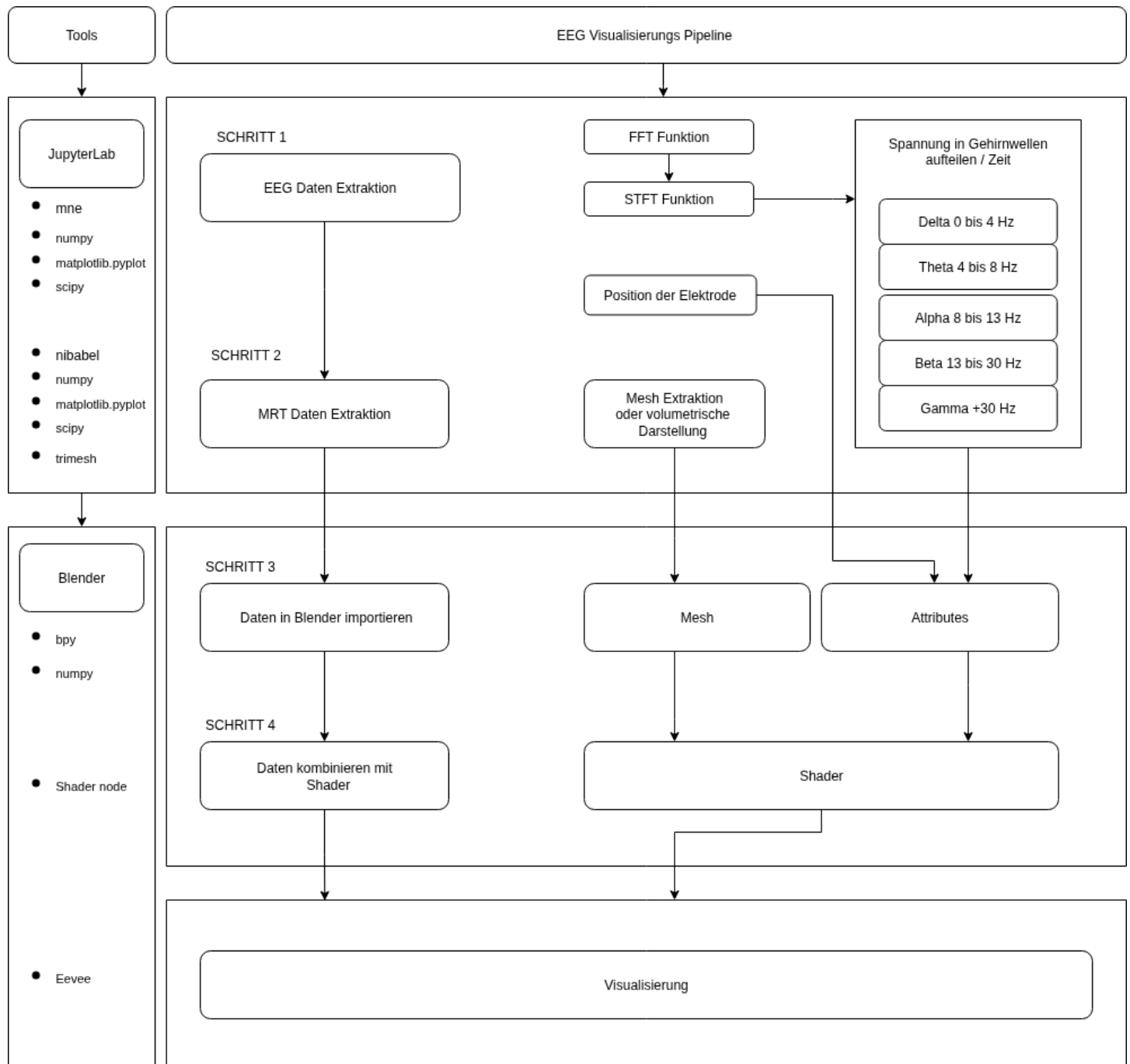


EEG Visualization

the idea: Visualising EEG signal/data in a more sense native way. mapping it on 3d mesh created from MRT data as along the normal displacement and heat map . this can provide the doctors with insight on where the the brain is mostly .this attempt dose not trying to replace to the traditional way of EEG reading is done rather as visual helper.

the pipe line :



Data Extraction:

Tools : jupyter labs

from the (.fif) file I used the MNE-Python library to extract the EEG channels voltage values

```
[4]: raw_eeg=raw.pick('eeg')
print("EEG channels:",len(raw_eeg.ch_names))
print ("channel name:",raw_eeg.ch_names)

EEG channels: 60
channel name: ['EEG 001', 'EEG 002', 'EEG 003', 'EEG 004', 'EEG 005', 'EEG 006', 'EEG 007', 'EEG 008', 'EEG 009', 'EEG 010', 'EEG 011', 'EEG 012', 'EEG 013', 'EEG 014', 'EEG 015', 'EEG 016', 'EEG 017', 'EEG 018', 'EEG 019', 'EEG 020', 'EEG 021', 'EEG 022', 'EEG 023', 'EEG 024', 'EEG 025', 'EEG 026', 'EEG 027', 'EEG 028', 'EEG 029', 'EEG 030', 'EEG 031', 'EEG 032', 'EEG 033', 'EEG 034', 'EEG 035', 'EEG 036', 'EEG 037', 'EEG 038', 'EEG 039', 'EEG 040', 'EEG 041', 'EEG 042', 'EEG 043', 'EEG 044', 'EEG 045', 'EEG 046', 'EEG 047', 'EEG 048', 'EEG 049', 'EEG 050', 'EEG 051', 'EEG 052', 'EEG 053', 'EEG 054', 'EEG 055', 'EEG 056', 'EEG 057', 'EEG 058', 'EEG 059', 'EEG 060']
```

In this project, I first applied FFT to inspect the overall frequency content of each electrode's signal[]

```
[6]: from numpy.fft import rfft, rfftfreq

sfreq = raw_eeg.info['sfreq']
freqs = rfftfreq(data.shape[1], d=1/sfreq)
fft_data = np.abs(rfft(data, axis=1))

print("Frequency resolution:", freqs[1], "Hz")
print("Max frequency:", freqs[-1], "Hz")
print("FFT shape:", fft_data.shape)

Frequency resolution: 0.00360008092939710736 Hz
Max frequency: 300.38749511718756 Hz
FFT shape: (60, 83401)
```

I then used STFT (Short-Time Fourier Transform) to decode each electrode's raw voltage signal into 5 different brain frequency channels (delta, theta, alpha, beta, gamma) over time, which FFT does not offer (it only calculates the frequency, but without the time value). This data is used as a float per vertex attribute to drive a heat map shader in Blender, as well as a normal based geometry displacement using Geometry Nodes.

masking the mesh (I used AI tools heavily for this Step)

After many attempts (I will mention them later), this approach was the best:

Mapping the electrodes to mesh vertices using distance weighting. Electrodes are split into left ($x < -0.10$), right ($x > 0.10$), and midline groups (midline electrodes are shared between both hemispheres). Using a KD-tree to find the 5 nearest electrodes for each mesh vertex, weights are computed with a Gaussian kernel, then I normalized them to add to 1 and applied a linear fade to so any area farther than 0.35 drop out of the mask

```
## AI coding help Start
from scipy.spatial import KDTree

lh_count = len(lh_verts_centered)
all_verts = np.vstack([lh_verts_centered, rh_verts_centered])

# Split electrodes by X position into LH, RH and midline
midline_tol = 0.10
lh_elec_mask = electrode_pos_scaled[:, 0] < -midline_tol
rh_elec_mask = electrode_pos_scaled[:, 0] > midline_tol
mid_elec_mask = ~lh_elec_mask & ~rh_elec_mask

lh_electrodes = electrode_pos_scaled[lh_elec_mask | mid_elec_mask]
rh_electrodes = electrode_pos_scaled[rh_elec_mask | mid_elec_mask]

print(f"LH electrodes: {lh_elec_mask.sum()}, RH electrodes: {rh_elec_mask.sum()}, Midline: {mid_elec_mask.sum()}")

# Distance mask per hemisphere using only its own electrodes
lh_tree = KDTree(lh_electrodes)
rh_tree = KDTree(rh_electrodes)

lh_dist, lh_nearest = lh_tree.query(lh_verts_centered, k=5)
rh_dist, rh_nearest = rh_tree.query(rh_verts_centered, k=5)

# Smooth fade zone instead of hard cutoff
fade_start = 0.25
fade_end = 0.35

lh_falloff = np.clip((fade_end - lh_dist[:, 0]) / (fade_end - fade_start), 0, 1)
rh_falloff = np.clip((fade_end - rh_dist[:, 0]) / (fade_end - fade_start), 0, 1)

# Gaussian interpolation between electrodes
sigma = 0.15
lh_weights = np.exp(-lh_dist**2 / (2 * sigma**2))
lh_weights = lh_weights / lh_weights.sum(axis=1, keepdims=True)
lh_weights *= lh_falloff[:, np.newaxis]

rh_weights = np.exp(-rh_dist**2 / (2 * sigma**2))
rh_weights = rh_weights / rh_weights.sum(axis=1, keepdims=True)
rh_weights *= rh_falloff[:, np.newaxis]

# Map hemisphere-local electrode indices to global electrode array
lh_elec_indices = np.where(lh_elec_mask | mid_elec_mask)[0]
rh_elec_indices = np.where(rh_elec_mask | mid_elec_mask)[0]

nearest_electrodes = np.vstack([lh_elec_indices[lh_nearest], rh_elec_indices[rh_nearest]])
weights = np.vstack([lh_weights, rh_weights])

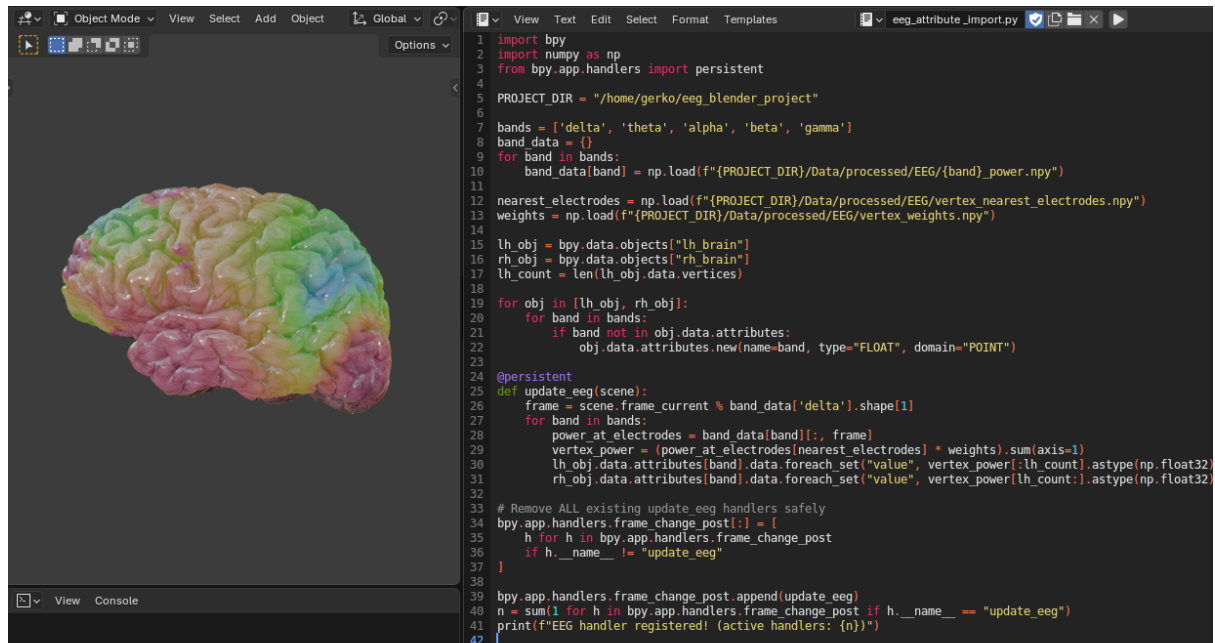
np.save(f'{PROJECT_DIR}/Data/processed/EEG/vertex_nearest_electrodes.npy', nearest_electrodes.astype(np.int32))
np.save(f'{PROJECT_DIR}/Data/processed/EEG/vertex_weights.npy', weights.astype(np.float32))
## AI coding help End

LH electrodes: 37 RH electrodes: 37 Midline: 6
```

DATA importing in Blender

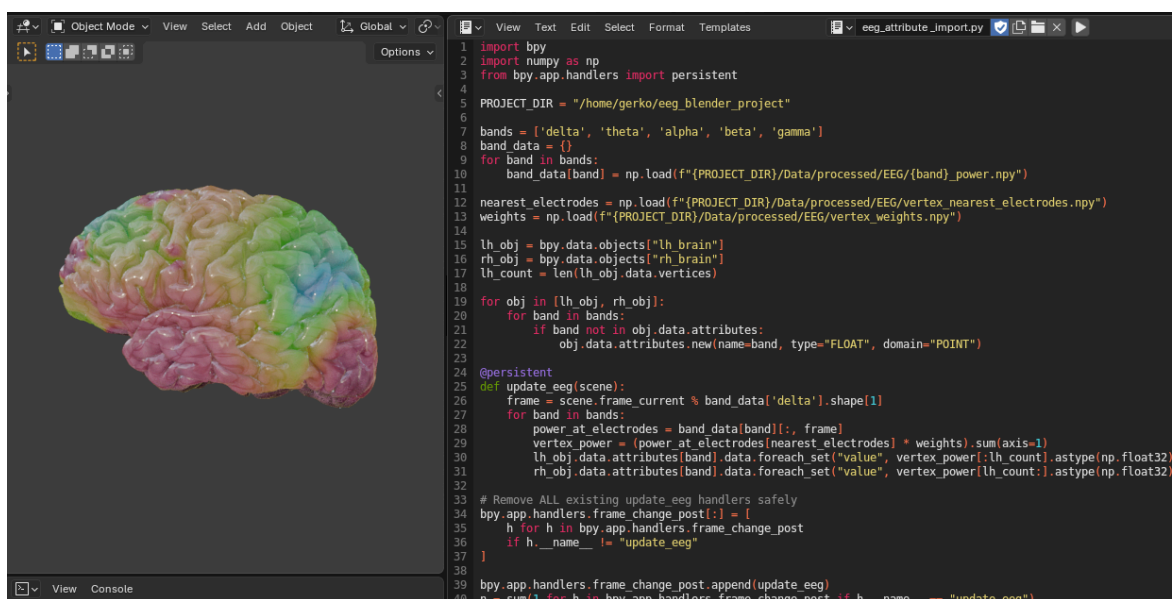
MRI

I used the data I extracted in the Jupyter Lab script to visualize the pial surface in Blender. I centered it and aligned it to the world axes.



EEG

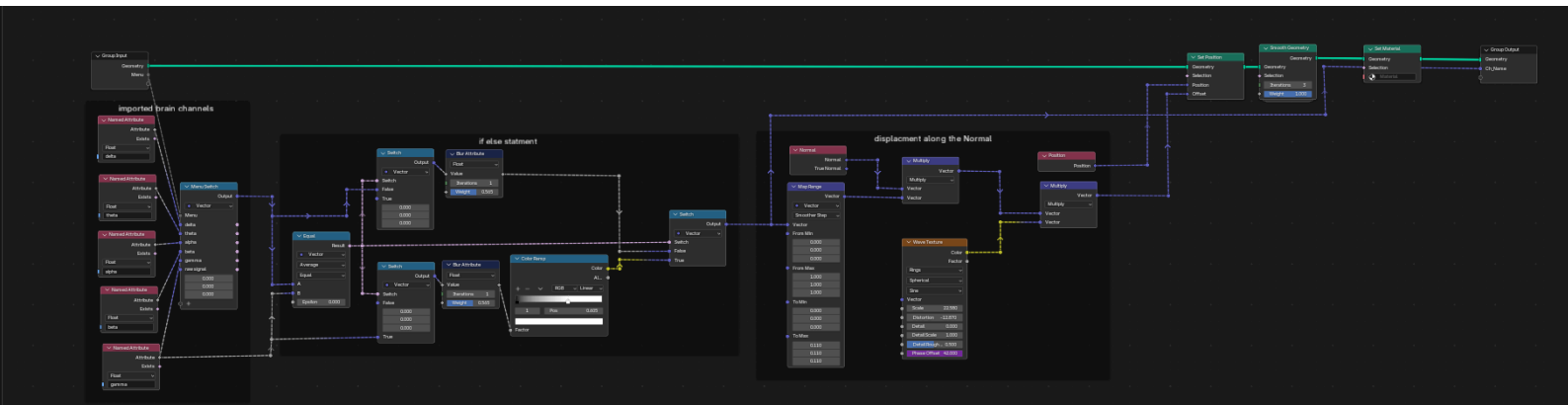
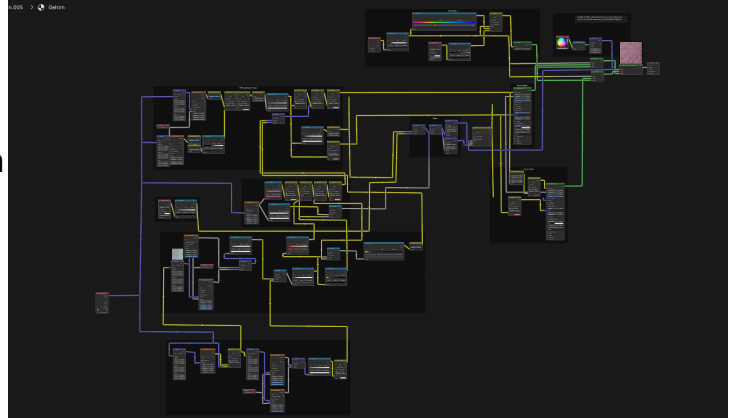
Then(`EEG_Attribute_Import.py`) stores the brain waves on the imported mesh's vertices as a per-vertex float field value that updates its value for each frame



Attributes in Shader/Geo Node

Shader node setup :

- 1) For the base brain texture, I used multiple Voronoi texture setups to build the albedo map, then i duplicate it modify and turned the saturation up to create the SSS map. I also built a noise texture setup to drive the roughness with it. I did build the displacement/bump map, but I did not connect it because it makes the animation really slow. (all of this was connected to a Principled BSDF node.)
- 2) For the heat map part, I used an Attribute node that used the attribute I exported from the geonode setup*** to drive the color and intensity of an Emission shader.



geo node setup:

I connected all imported attributes (brain channels) to a Menu node, then I used the output in:

- 1) Making a UI drop down list on the geo node modifier using the Group Input node.
- 2) Exporting a single value to pass it to the shader setup (I mentioned this before).***
- 3) Mapping the attribute to be combined with the normal, then displacing its position using the Wave texture (on a mesh level, this will do a scalar-matrix multiplication operation).

Tools and math I learned and used :

- Blender
- Jypeter lab
- FFT/FDT
- KD Tree
- numpy

what I think I did wrong which in a real world situation i will never / less do :

In the medical world, raw data matters and should rarely be altered or changed (because these changes can lead to less accuracy, missing data, etc., which is very dangerous (wrong diagnostics, wrong medication prescribing, life threatening).) So my project, for now, sits more or less in the middle between an artistic approach (motion design) and medical visualization.

things I did :

- Normalized data
- Lowered the mesh resolution (less accuracy)
- Guessed values
- Used mesher rather than volumetric reconstruction of the MRI (so my laptop does not hunt me down)
-

things I tried and didn't go well :

I faced a very big challenge with mapping the voltage value to the correct mesh region (masking the active region).

list of the approaches before the one I went with it :

- 1) normals pointing toward the inner part of the brain hemisphere (x, -x depending on the part) and also the z-axis. This would work if the brain surface were a star shaped convex, but because the mesh has a lot of polygons with normals pointing all over the place, this approach didn't deliver a clean mask.
- 2) fixed threshold from the most middle point existing on the mesh. That left a very harsh cut, and because the brain has folds, lowering the threshold was not an option.
- 3) volume-based masking, but this was too heavy — with 312K vertices, but I was fast out of RAM (I did not try it on the 30% mesh this can be a more faster and accurate)
- 4-) Shrinking a sphere around the brain hemispheres, baked the change into vertex positions, and then moving it toward the middle part as well to the bottom. I used the intersection as a mask border, where the inner part is positive and the outer part is negative mask.

Resources I (stop_on,used,learned_from){

- ⊕ Alexandre Savio - Nipy on functional brain MRI
 - ⊕ Neuroimaging: Visualize 3D MRI Brain Scans with Python
 - ⊕ L8: MNE tutorial Part #1 - Load and Segment continuous EEG data
 - ⊕ How to import Medical scans into Blender 3.0 ,
 - ⊕ Import CSV data into Blender
 - ⊕ Neurohackademy: Chris Markiewicz - NiBabel: Neuroimaging data and file structures in P...
 - ⊕ L9: MNE tutorial part #2 - EEG/ERP Visualization and Time-Frequency Analysis
 - ⊕ L10: Overview of artifact and noise detection in EEG/MEG
 - ⊕ Learn NumPy in 1 hour!  ,
 - ⊕ What is a Gaussian Distribution?
 - ⊕ K-d Trees - Computerphile ,
 - ⊕ But what is the Fourier Transform? A visual introduction.
 - ⊕ Wavelets: a mathematical microscope
 - ⊕ The short-time Fourier transform (STFFT)
 - ⊕ The Fast Fourier Transform (FFT): Most Ingenious Algorithm Ever?
 - ⊕ Digital Music Module 11: Short-Time Fourier Transform (STFT) Chromatic And Vibrato
 - ⊕ RBF Kernel Explained: Mapping Data to Infinite Dimensions
 - ⊕ K-d Tree in Python #4
 - ⊕ Example Gaussian Filter
 - ⊕ SciPy - A Scientific Extension of NumPy | Python
 - ⊕ Matplotlib Full Python Course - Data Science Fundamentals ⊕ W3Schools.com
 - ⊕ Normal distribution
 - ⊕ fourier/Fourier Transform - A Visual Introduction.ipynb at master · thatSaneKid/fourier
 - ⊕ Fade (audio engineering)
- <https://matplotlib.org/stable/tutorials/index>
- <https://docs.scipy.org/doc/scipy/>
- ⊕ Flowchart Maker & Online Diagram Software ,
 - ⊕ Blender - The Free and Open Source 3D Creation Software — blender.org
 - ⊕ Canva: Visual Suite for Everyone
 - ⊕ Project Jupyter
- <https://mne.tools/stable/api/datasets.html#module-mne.datasets>

Ai {

www.google.de ,informationgathering ,resursching
claude.ai + [deepseek](https://deepseek.com) Brainstorming+debuging

I commented the code, which was built mostly using AI. Just to be clear every concept the AI used, I looked up and learned an read .

}

}